Experiment 1: Creating and Using a User-Defined Package

Aim:
To create a user-defined package and access its class from another Java program.

Tasks:

1. Create a package named `mypackage`.
2. Create a class `Student` inside the package.
3. Define student details and a method to display them.
4. Create another class outside the package.
5. Import and use the `Student` class.

Experiment 2: Importing Packages

Aim:
To demonstrate different ways of importing and accessing classes from a package.

Tasks:

1. Create a package `college`.
2. Create classes `Student` and `Faculty`.
3. Import a specific class from the package.
4. Demonstrate wildcard import using `college.*`.
5. Access the classes from another program.

Experiment 3: Packages and Member Access

Aim:
To demonstrate the accessibility of class members across packages using access modifiers.

Tasks:
Create a class containing:

- `public` member
- `private` member
- `protected` member
- default member

Access these members from:

1. The same class
2. Another class in the same package
3. A class in a different package

Experiment 4: Demonstrate CLASSPATH

Aim:
To demonstrate how Java uses the `CLASSPATH` to locate user-defined packages and classes.

Tasks:

1. Create a package `utilities`.
2. Create a class `Calculator`.
3. Compile the package.
4. Set the `CLASSPATH`.
5. Write another Java program that uses the `Calculator` class.
6. Compile and execute the program using the configured `CLASSPATH`.

Experiment 5: Package Containing Multiple Classes

Aim:
To create and use a package containing multiple classes.

Tasks:
Create a package named `bank` containing:

- `Account`
- `Customer`
- `Transaction`

Create a main class outside the package and access all three classes.

INTERFACES

Experiment 6: Defining and Implementing an Interface

Aim:
To create an interface and implement its methods in a class.

Problem Statement:
Create an interface `Shape` containing a method `area()`. Implement the interface using `Circle` and `Rectangle` classes.

Experiment 7: Implementing Multiple Interfaces

Aim:
To demonstrate how a class can implement multiple interfaces.

Problem Statement:
Create two interfaces:

```
Printable
    |
    +-- print()

Showable
    |
    +-- show()
```

Create a class `Demo` that implements both interfaces.

Expected Output:

```
Printing...
Showing...
```

Experiment 8: Interface-Based Polymorphism

Aim:
To demonstrate polymorphism using an interface reference.

Problem Statement:
Create an interface `Vehicle` with a method `start()`. Implement it using `Car` and `Bike`.

Use:

```
Vehicle v;
```

and assign objects of both `Car` and `Bike` to the reference.

Experiment 9: Variables in Interfaces

Aim:
To demonstrate variables declared inside an interface.

Problem Statement:
Create an interface `Constants` containing constants such as:

```
int MAX_MARKS = 100;
double PI = 3.14159;
```

Access these variables from a class implementing the interface.

Tasks:

1. Declare variables in an interface.
2. Access them using the interface name.
3. Demonstrate that interface variables are constants.

Experiment 10: Nested Interfaces

Aim:
To demonstrate the concept of nested interfaces.

Problem Statement:
Create an outer class `University` containing a nested interface `Department`.

Implement the nested interface in another class and demonstrate its functionality.

Experiment 11: Interface Inheritance

Aim:
To demonstrate inheritance between interfaces.

Problem Statement:

Create:

```
interface Animal
        |
        | extends
        ↓
interface Dog
        |
        | implemented by
        ↓
    Labrador
```

`Animal` should contain `eat()`, while `Dog` should contain `bark()`.

Implement both methods in `Labrador`.

Experiment 12: Real-World Application Using Interfaces

Aim:
To develop a Java application using interfaces.

Problem Statement:
Develop a **Payment Processing System** using an interface `Payment`.

The interface should contain:

```
void pay(double amount);
```

Implement it using:

- `CreditCardPayment`

- `UPIPayment`
- `NetBankingPayment`

The main program should allow the user to select a payment method and process the payment.

Concepts Covered:

- Interfaces
- Implementing interfaces
- Polymorphism
- Abstraction
- Multiple implementations
- Real-world application of interfaces